A DEVELOPER'S DISCOVERY

I Run Every Commit Through 5 AI Reviewers. Here's What They Caught.

How a solo developer iterated from "review this" to five independent AI personas catching real bugs before they ship.

Claude Code | Subagents | Code Review | Solo Dev | Rust

Lim Chun Zhe

THE PROBLEM

Solo Dev.
No Code Review.

I'm building a Rust project solo. No team. No PR reviews.
The bugs I was shipping had a pattern.

BUGS THAT SHIPPED

❌ **Config typo silently swallowed**
A wildcard match caught all strings. Misspell a retriever name and it just... works, doing nothing.

❌ **CLI returned success on failure**
Server unreachable? Exit code 0. Looked like success in every CI pipeline.

❌ **Race condition in log rotation**
Size check was outside the mutex. A TOCTOU race that passed every test.

❌ **DoS vector in dashboard**
No file size limit on log reads. One large file could OOM the server.

These aren't edge cases. They're the kind of bugs that pass every test and break in production. The kind a teammate would catch in a PR review.

THE ITERATION

From "Review This" to Five Personas

Each iteration caught things the previous one couldn't.

V1: "Review this diff"
Generic feedback. Everything is "looks good" with surface-level suggestions.
GENERIC

↓

V2: One reviewer with a role
"You are a senior SWE." Immediately better. Started flagging real bugs.
FOCUSED

↓

V3: Three independent subagents
SWE + PM + QA. Separate context windows. Don't see each other's feedback.
MULTI-LENS

↓

V4: Five personas with dynamic slots
3 standard + 2 rotate based on what changed. Security for auth. Rust Idiom for types.
CURRENT

THE SYSTEM

Five Independent Reviewers

Each runs as a separate subagent. Own context window. Cites specific file:line references. Gives a verdict.

ALWAYS PRESENT

💻

Senior SWE

Correctness, safety, race conditions, edge cases, performance

📋

Product Manager

Goal alignment, scope creep, incomplete work, UX coherence

🔍

QA Engineer

Test coverage, failure modes, boundaries, regression risk

DYNAMIC: AI CREATES 2 PER REVIEW

No fixed list. Claude analyzes the changed files and **invents the two most relevant reviewer personas on the fly**. Some examples it has spawned:

- 🔒 Security Engineer
- ⚡ Performance Engineer
- 🦀 Rust Idiom Reviewer
- 🐘 Memory Systems
- 🎨 UX Designer
- 📊 API Design Reviewer

The pool is unbounded. Touch a new domain and Claude creates a new specialist. That's the point. You get expertise you didn't think to ask for.

REAL EXAMPLES

What Each Reviewer Actually Catches

💻 Senior SWE

TOCTOU race in log rotation • Wildcard match swallowing config typos • CLI exit 0 on server failure • HNSW queries hitting partially-built index during rebuild

📋 Product Manager

Blocking I/O fix applied to 2 of 7 call sites (incomplete work shipped as "done") • Scope creep in design docs • Missing user-facing documentation

🔎 QA Engineer

NaN/infinity propagation in scores with no guards • u32 overflow in access counts • Missing boundary tests • Stale build cache when env var changes

🔄 Dynamic Reviewers (created on the fly)

Claude spawned a Rust Idiom reviewer that replaced 3 pin fields with Option<PinInfo> • A Security reviewer it created caught session logs leaking prompt secrets • A Performance reviewer turned O(S×E) → O(E) in a GC cycle

The SWE finds bugs. The QA finds missing tests. The PM finds incomplete work. The dynamic reviewers find what none of the above would think to look for.

WHAT THE OUTPUT LOOKS LIKE

Real Review Output: HNSW Index Rebuild

Actual findings from a single 5-persona review, triaged by severity.

```
CRITICAL (blocks merge)
  Rebuild serves partial index during build. Not true blue-green
  next_hnsw_id reset after rebuild start. Race window

IMPORTANT (blocks merge)
  No concurrency tests for search-during-rebuild
  Missing HNSW save/load roundtrip test

LOW (must fix)
  flush_hnsw pins Arc during long I/O
  Old index deallocation blocks tokio runtime
  No swap event logged for observability
```

2	2	3
Critical	Important	Low

Both criticals were real bugs. Without this review, queries would have hit a half-built index during rebuilds. All 7 findings were fixed in a single commit.

THE NUMBERS

1 in 8 Commits Is a Review Fix

Real numbers from the git history of a Rust knowledge-base project.

323
Total commits

42
Review fix commits

13%
Commits are review fixes

23+
Design issues caught in specs

WHAT 42 FIXES LOOK LIKE

Race conditions, silent error swallowing, scoring bugs, DoS vectors, incomplete implementations, missing migrations, API design improvements. Every one of these would have shipped without review.

Each reviewer must produce 10+ lines of substantive feedback with file:line references. Shallow reviews get re-dispatched.

NOT JUST CODE

Reviews Run After Design Too

The same five lenses review specs before a line of code is written. Fixing a design is minutes. Fixing an implementation is hours.

❌

Code-Only Review

- Catch bugs after writing them
- Architectural flaws baked in
- Scope creep discovered late
- Fix cost: hours to days

✅

Design + Code Review

- Catch flaws before coding
- Architecture validated early
- Scope agreed upfront
- Fix cost: minutes

CAUGHT IN DESIGN REVIEW, NOT CODE

Knowledge Lifecycle RFC

2-round review by Memory Systems + Performance reviewers generated **13 implementation issues** from the spec. Zero code existed yet.

Telemetry Traces Spec

Review found **10 issues**: missing data contracts, wrong module mapping, incompatible CSS approach. All fixed in the doc, not the code.

Session History Spec

Wrong propagation site caught. Implementation would have gone in the wrong file entirely. **Hours of wasted work avoided.**

KEY TAKEAWAYS

What I Learned

🎯 **Roles change everything**

A generic "review this" catches surface issues. A PM persona catches incomplete work. A Security persona catches leaked secrets. The role defines what gets found.

📈 **Independent context windows matter**

One reviewer with five roles collapses to one perspective. Five separate subagents find different things because they think independently.

📄 **Review designs, not just code**

The highest-leverage reviews happen before implementation. A flaw caught in a spec costs minutes. In code, it costs hours.

⚡ **Speed without review is shipping bugs faster**

AI makes writing code fast. The real leverage is simulating the team dynamics that catch mistakes before they ship.

What's the best bug your AI reviewer caught that you would have shipped?

Curious what setups people have landed on.